Chuleta Python para el examen

2º DAW · Programación · Temas 1-6 · Python 3.12. **Regla de oro:** el máximo solo se da si usas la forma que pide el enunciado (recuadros amarillos).

Índice

1. Entorno
2. Operadores
3. Condicionales
4. Bucles
5. Funciones
6. Clases y POO
7. Listas de objetos
8. Diccionarios
9. Comprensiones y lambda
10. Métodos de str y f-strings
11. Métodos de list
12. Métodos de dict
13. Conversión de tipos
14. Java → Python
15. Resumen: formas exigidas

1 Entorno

Ejecutar un programa

```python
# hola_mundo.py
print("¡Hola, Mundo!")
```

```
python hola_mundo.py      # Windows / macOS
python3 hola_mundo.py     # Linux / Ubuntu / Docker
```

Esqueleto de cualquier archivo

```python
"""Docstring del módulo: qué hace el programa."""


def main():
    """Punto de entrada."""
    print("Hola")


if __name__ == "__main__":
    main()
```

Si me piden… → uso…

Si me piden…	Uso…
Abrir terminal en VS Code	`Ctrl+ñ`

Ejecutar en Linux	`python3 archivo.py`
Mostrar algo	`print("texto", variable)`
Leer del teclado	`input("Mensaje: ")` → **siempre str**
Comentario de una línea	`# comentario`
Documentar función/clase	`"""docstring"""` justo debajo del `def` / `class`
Saber el tipo	`type(x)`
Buscar en todos los archivos	`Ctrl+Shift+F` (VS Code)

PEP 8 en 5 reglas

Sangría	**4 espacios** (nunca mezclar con tabuladores)
Variables y funciones	`snake_case` : `precio_total`
Clases	`PascalCase` : `CuentaBancaria`
Constantes	`MAYUSCULAS` : `IVA = 0.21`
Espacios	`x = a + b` , tras comas; 2 líneas en blanco entre funciones

> ★ **FORMA QUE EXIGE EL PROFESOR**
>
> - En Linux/Docker se ejecuta con `python3` .
> - Código PEP 8, docstrings con triple comilla en funciones y clases, y las pruebas dentro de `if __name__ == "__main__":` .
> - Los proyectos en Linux se guardan en `~/` (p. ej. `~/proyectos_clase`), nunca en `/mnt/c/...` .

> ✗ **Errores típicos**
>
> - `IndentationError` : mezclar tabuladores y espacios, o sangrar sin motivo.
> - Olvidar los **dos puntos** `:` al final de `if` , `for` , `def` , `class` … → `SyntaxError` .
> - Escribir `python` en Ubuntu y que no exista: usa `python3` .

2 Operadores

Aritméticos

```
7 + 2    # 9
7 - 2    # 5
7 * 2    # 14
7 / 2    # 3.5   ¡siempre float!
7 // 2   # 3     división entera
7 % 2    # 1     resto (módulo)
2 ** 3   # 8     potencia
x += 1   # no existe x++
```

Comparación, lógicos, pertenencia, identidad

```
==  !=  <  >  <=  >=
and  or  not          # NO && || !
"a" in "casa"         # True
3 not in [1, 2]       # True
lista1 == lista2      # mismo VALOR
lista1 is lista3      # mismo OBJETO en memoria
x is None             # forma correcta de mirar None
```

Si me piden... → uso...

Si me piden...	Uso...
¿Es par?	`n % 2 == 0`
¿Es múltiplo de 3?	`n % 3 == 0`
Cociente entero	`a // b`
Último dígito	`n % 10`
Raíz cuadrada	`x ** 0.5`
Comparar contenido	`==`
¿Es el mismo objeto?	`is` / `is not`
¿Está dentro?	`in` / `not in`
Redondear a 2 decimales	`round(x, 2)` o `f"{x:.2f}"`

Valores "falsos" (falsy)

`False`, `None`, `0`, `0.0`, `""`, `[]`, `{}` → en un `if` cuentan como falso. Todo lo demás es verdadero.

★ FORMA QUE EXIGE EL PROFESOR

- Paridad con el operador módulo: `if numero % 2 == 0:`
- `==` para comparar valor e `is` / `is not` para identidad. En Python **no existe** `===`.

✗ Errores típicos

- `=` (asignar) en vez de `==` (comparar) dentro de un `if` → `SyntaxError`.
- Esperar que `7 / 2` dé `3`: da `3.5`. Para entero usa `//`.
- Escribir `&&`, `||`, `!` o `i++` por costumbre de Java.
- `"5" + 3` → `TypeError`: convierte antes con `int()` o `str()`.

3 Condicionales

if / elif / else

```
nota = 75
if nota >= 90:
    print("Excelente")
elif nota >= 75:
    print("Muy bien")
elif nota >= 60:
    print("Bien")
else:
    print("Insuficiente")
```

Comparación encadenada (rango)

```
temp = 22.5
if 18.0 <= temp <= 27.0:
    print("La temperatura es agradable.")
else:
    print("Hace demasiado frío o demasiado calor.")
```

Ternario (una línea)

```
msg = "mayor" if edad >= 18 else "menor"
```

match-case (el "switch" de Python)

```
match dia_numero:
    case 1:
        print("Lunes")
    case 2 | 3:                 # varias opciones
        print("Martes o miércoles")
    case _:                     # comodín = default
        print("Día no válido")
```

match con guardas

```
match peso:
    case p if p <= 0:
        print("Peso no válido.")
    case p if p <= 5:
        print("Paquete pequeño.")
    case p if 5 < p <= 20:
        print("Paquete mediano.")
    case _:
        print("Paquete grande.")
```

Cláusula de guarda (return temprano)

```
def calcular_raiz_cuadrada(x):
    """Devuelve la raíz de x si es positivo."""
    if x <= 0:
        print("Error: Por favor, use solo números positivos.")
        return                  # sale ya, sin else
    resultado = x ** 0.5
    return resultado
```

Si me piden...	Uso...
Varios casos excluyentes por rangos	`if / elif / else` (de mayor a menor)
Menú o valores concretos (1, 2, 3...)	`match valor:` + `case 1:` + `case _:`
Rangos dentro de un match	guardas `case p if p <= 5:`
¿Está entre A y B?	`A <= x <= B`
Validar error al principio de una función	guarda: `if error: ... return`

★ FORMA QUE EXIGE EL PROFESOR

- **match-case** con comodín `case _` obligatorio; si hay rangos, **guardas** `case p if condición`.
- Rangos con **comparación encadenada** `18.0 <= temp <= 27.0`. PROHIBIDO `temp >= 18.0 and temp <= 27.0`.
- **Cláusula de guarda** con `return` temprano en vez de `if/else` anidados ("código en flecha").

✗ Errores típicos

- Olvidar `:` tras `if`, `elif`, `else`, `case`.
- Escribir `else if` (Java) en vez de `elif`.
- Ordenar mal los `elif`: si pones `nota >= 40` primero, nunca llega a "Excelente".
- Comparar un `input()` sin convertir: `"20" >= 18` → `TypeError`.
- `match` sin `case _`: si nada coincide no pasa nada y pierdes puntos.

4 Bucles

for sobre lista, cadena y range

```
total = 0
for precio in [15.50, 20.0, 35.75]:
    total += precio            # acumulador (sin sum)

for letra in "hola":
    print(letra)

range(5)          # 0 1 2 3 4
range(1, 11)      # 1 … 10   (el final NO entra)
range(20, -1, -2) # 20 18 … 0 (cuenta atrás)
```

enumerate (numerar desde 1)

```
tareas = ["Comprar leche", "Estudiar Python"]
for i, tarea in enumerate(tareas, start=1):
    print(f"{i}. {tarea}")
```

Diccionario con .items()

```
inventario = {"manzanas": 50, "peras": 30}
for fruta, cantidad in inventario.items():
    print(f"Hay {cantidad} {fruta}")
```

Anidados

```
for talla in ["M", "L"]:
    for color in ["Rojo", "Azul"]:
        print(talla, color)
```

while, break, continue

```
clave = input("Contraseña: ")
while clave != "python123":
    print("Acceso denegado")
    clave = input("Contraseña: ")   # ¡actualizar!

total = 0
while True:                         # infinito controlado
    dato = input("Número o 'salir': ")
    if dato == "salir":
        break                       # sale del bucle
    total += int(dato)

for n in range(1, 11):
    if n % 3 == 0:
        continue                    # salta este n
    print(n)
```

else del bucle (si NO hubo break)

```
for n in [10, 20, 30]:
    if n == 99:
        print("¡Encontrado!")
        break
else:
    print("Número no encontrado")
```

Leer un número validando

```
texto = input("Edad: ")
while not texto.isdigit():
    texto = input("Escribe un entero: ")
edad = int(texto)
```

Si me piden...	Uso...
Recorrer una lista / cadena	`for elem in lista:`
Repetir N veces / tabla de multiplicar	`for i in range(1, 11):`
Pares de 20 a 0	`range(20, -1, -2)`
Numerar desde 1	`enumerate(lista, start=1)`
Repetir hasta que el usuario acierte	`while condición:`
Repetir hasta escribir "salir"	`while True:` + `if ...: break`
Saltar algunos elementos	`continue`
Buscar y avisar si no está	`for ... break` + `else:`
Máximo de intentos	`while intentos < 3 and not acertado:`
Contar / sumar	`contador += 1` / `total += x` (iniciados a 0 fuera)

★ FORMA QUE EXIGE EL PROFESOR

- Suma del carrito **iterando**: PROHIBIDO `sum()`.
- Tabla de multiplicar con `range()`; cuenta atrás con **paso** (`range(20, -1, -2)`).
- Numerar con `enumerate(tareas, start=1)`.
- Diccionarios con `.items()` (o `.values()`).
- Buscar el 99 con el `else` **del bucle**.
- Calculadora con `while True` + `break`; saltar múltiplos de 3 con `continue`.
- Cubos / filtrar palabras: **comprensión de listas en una sola línea** (ver comprensiones).

✗ Errores típicos

- **while infinito**: no actualizar la variable de la condición dentro del bucle.
- `range(1, 10)` llega solo hasta 9: el final no se incluye.
- Iniciar el acumulador **dentro** del bucle (se reinicia en cada vuelta).
- `for i in range(len(lista) + 1)` → `IndexError`.
- Borrar de una lista mientras la recorres: recorre una copia `lista[:]` o usa una comprensión.
- El `else` del bucle va alineado con el `for`, no con el `if`.

5 Funciones

Definir, devolver, valores por defecto

```
def multiplicar(a, b):
    """Devuelve a por b (sin imprimir)."""
    return a * b

resultado = multiplicar(3, 4)        # 12

def presentar_usuario(nombre, rol="Estudiante"):
    """Imprime nombre y rol."""
    print(f"Usuario: {nombre} | Rol: {rol}")

presentar_usuario("Ana")
presentar_usuario("Luis", "Profesor")

def es_mayor_de_edad(edad):
    """True si edad >= 18."""
    return edad >= 18                # sin if/else
```

Devolver varios valores (tupla)

```
def obtener_extremos(lista):
    """Devuelve (mínimo, máximo)."""
    return min(lista), max(lista)

minimo, maximo = obtener_extremos([4, 8, 1])
```

Type hints + docstring

```
def celsius_a_fahrenheit(celsius: float) -> float:
    """Convierte grados Celsius a Fahrenheit."""
    return (celsius * 9 / 5) + 32
```

*args y **kwargs

```python
def calcular_promedio(*args):
    """args es una TUPLA con todos los números."""
    return sum(args) / len(args)

calcular_promedio(4, 6, 8)           # 6.0

def crear_perfil_usuario(nombre, apellido, **kwargs):
    """kwargs es un DICCIONARIO con el resto."""
    perfil = {"nombre": nombre, "apellido": apellido}
    perfil.update(kwargs)
    return perfil

crear_perfil_usuario("Ana", "Ruiz", edad=20, ciudad="Adeje")
```

Decorador

```python
def anunciar_ejecucion(funcion):
    """Añade mensajes antes y después."""
    def envoltura(*args, **kwargs):
        print("--- Iniciando función ---")
        resultado = funcion(*args, **kwargs)
        print("--- Función terminada ---")
        return resultado
    return envoltura

@anunciar_ejecucion
def tarea_importante():
    print("Procesando datos en la base de datos...")

tarea_importante()
```

Orden superior y recursividad

```python
def aplicar_operacion(a, b, funcion):
    return funcion(a, b)

aplicar_operacion(5, 3, sumar)       # sin () en sumar

def calcular_factorial(n):
    if n <= 1:                       # caso base
        return 1
    return n * calcular_factorial(n - 1)
```

Si me piden...	Uso...
"Retorna / devuelve"	`return` (y **no** `print` dentro)
"Imprime / muestra"	`print` dentro de la función
Parámetro con valor por defecto	`def f(x, rol="Estudiante"):`
Cantidad indefinida de números	`*args` (tupla)
Cualquier dato con nombre	`**kwargs` (diccionario) → `for k, v in kwargs.items():`
Varios resultados	`return a, b` → `x, y = f()`
Anotaciones de tipo	`def f(x: float) -> float:`
Modificar una función sin tocarla	decorador + `@nombre`
Lista por defecto	`def f(lista=None):` + `if lista is None: lista = []`

★ FORMA QUE EXIGE EL PROFESOR

- Si dice "retorna": `return` obligatorio y PROHIBIDO `print()` dentro.
- `es_mayor_de_edad` : `return edad >= 18` directamente (sin if/else redundante).
- `obtener_extremos` : `return min(lista), max(lista)` y **desempaquetar** en dos variables.
- `*args` y `**kwargs` cuando lo pida; type hints `(celsius: float) -> float` + docstring.
- Decorador aplicado con `@anunciar_ejecucion` .
- `map` + `lambda` : `list(map(lambda x: x**2, numeros))` .
- PROHIBIDO mutables por defecto (`lista=[]` , `dic={}`): usa `None` .

✗ Errores típicos

- Olvidar `return` → la función devuelve `None` .
- `return` **dentro** del bucle: sale en la primera vuelta. Ponlo fuera (menos sangría).
- Llamar sin paréntesis: `saludar` en vez de `saludar()` .
- Recursividad sin caso base → `RecursionError` .
- Definir la función **después** de llamarla → `NameError` .

6 Clases y POO

Clase completa (molde del examen)

```
class PersonajeTeletubby:
    """Representa a un Teletubby."""

    saludo_oficial = "¡Tubby abrazo!"     # atributo de CLASE

    def __init__(self, nombre, color, antena, objeto_favorito):
        """Constructor: atributos de INSTANCIA."""
        self.nombre = nombre               # público
        self.color = color
        self.antena = antena
        self.objeto_favorito = objeto_favorito
        self.__nivel_hambre = 50           # privado (__)
        self.__nivel_felicidad = 100

    def jugar(self):
        """Juega: sube hambre y felicidad."""
        print(f"{self.nombre} juega con {self.objeto_favorito}")
        self.__nivel_hambre += 20
        self.__nivel_felicidad += 10

    def ver_estado(self):                  # getter
        """Devuelve el estado como texto."""
        return (f"Felicidad: {self.__nivel_felicidad} | "
                f"Hambre: {self.__nivel_hambre}")

    def __str__(self):                     # para el usuario
        # formato del enunciado: Teletubby Tinky Winky (Morado) - Feliz: 100
        return (f"Teletubby {self.nombre} ({self.color}) - "
                f"Feliz: {self.__nivel_felicidad}")

    def __repr__(self):                    # técnico
        return (f"PersonajeTeletubby('{self.nombre}', '{self.color}', "
                f"'{self.antena}', '{self.objeto_favorito}')")
```

Getters / setters con validación

```
class Producto:
    """Producto con precio privado."""

    def __init__(self, nombre, precio):
        self.nombre = nombre
        self.__precio = precio

    def get_precio(self):
        """Getter: devuelve el precio."""
        return self.__precio

    def set_precio(self, nuevo):
        """Setter: solo acepta precios positivos."""
        if nuevo <= 0:
            print("Precio no válido")
            return
        self.__precio = nuevo
```

Usar la clase

```
if __name__ == "__main__":
    po = PersonajeTeletubby("Po", "Rojo", "Círculo", "Patinete")
    po.jugar()                      # método: con ()
    print(po.ver_estado())
    print(po)                       # usa __str__
    print(PersonajeTeletubby.saludo_oficial)
    # po.__nivel_hambre  -> AttributeError (name mangling:
    # internamente se llama po._PersonajeTeletubby__nivel_hambre)
```

Herencia y super()

```
class Animal:
    def __init__(self, nombre):
        self.nombre = nombre

class Perro(Animal):
    def __init__(self, nombre, raza):
        super().__init__(nombre)    # llama al padre
        self.raza = raza

# Herencia múltiple: Cyborg(Humano, Robot); Cyborg.mro()
```

Si me piden...	Uso...
Dato compartido por todos los objetos	atributo de clase (fuera de `__init__`)
Dato propio de cada objeto	`self.x = x` dentro de `__init__`
Atributo privado	`self.__x` (dos guiones bajos)
Leer / cambiar un privado	getter `get_x()` / setter `set_x(valor)`
Que `print(obj)` salga bonito	`__str__` (devuelve str, no imprime)
Que la lista de objetos se imprima legible	`__repr__`
Reutilizar el constructor del padre	`super().__init__(...)`

★ FORMA QUE EXIGE EL PROFESOR

- Nombre de clase en **PascalCase**; docstring en la clase y en cada método.
- Atributo de clase (`saludo_oficial`) + atributos de instancia públicos en `__init__` + privados con `__` (name mangling).
- Getter que **devuelve** (`return`) una cadena con el estado; `__str__` amigable y `__repr__` técnico.
- Pruebas en `if __name__ == "__main__":` con al menos dos objetos.
- Teoría: si haces `po.__nivel_hambre = 0` fuera, **no** cambias el privado (creas otro atributo nuevo); leerlo da `AttributeError`. Se usan privados para proteger el estado y obligar a cambiarlo con métodos.

✗ Errores típicos

- Olvidar `self` como primer parámetro → `TypeError: ... takes 0 positional arguments but 1 was given` .
- Escribir `nombre = nombre` en vez de `self.nombre = nombre` (no se guarda).
- `__init__` con un solo guion bajo (`_init_`) → el constructor no se ejecuta.
- `__str__` que hace `print` en vez de `return` → `TypeError` .
- Lista mutable como atributo de clase (`trucos = []`): la comparten todos. Va en `__init__` .

7 Listas de objetos

```python
class Tienda:
    """Gestiona una lista de productos."""

    def __init__(self):
        self.productos = []                    # lista vacía

    def anadir(self, producto):
        self.productos.append(producto)

    def listar(self):
        for i, p in enumerate(self.productos, start=1):
            print(f"{i}. {p}")

    def buscar(self, nombre):
        for p in self.productos:
            if p.nombre.lower() == nombre.lower():
                return p
        return None                            # fuera del for

    def eliminar(self, nombre):
        p = self.buscar(nombre)
        if p is None:
            return False
        self.productos.remove(p)
        return True
```

```python
# Filtrar (comprensión)
baratos = [p for p in tienda.productos if p.get_precio() < 10]

# Ordenar sin cambiar la original
por_precio = sorted(tienda.productos, key=lambda p: p.get_precio())
# Ordenar la propia lista por nombre, de la Z a la A
tienda.productos.sort(key=lambda p: p.nombre, reverse=True)

# Mayor / menor por un atributo
caro = max(tienda.productos, key=lambda p: p.get_precio())

# Total sumando un atributo
total = sum(p.get_precio() for p in tienda.productos)

# Lista de nombres
nombres = [p.nombre for p in tienda.productos]

print(tienda.productos)   # usa __repr__ de cada objeto
```

Si me piden...	Uso...
Guardar objetos	`self.lista = []` + `append(obj)`
Listar numerado	`enumerate(lista, start=1)`
Buscar por atributo	`for` + `if` + `return obj` ; `return None` al acabar
Filtrar	`[o for o in lista if condición]`
Ordenar por atributo	`sorted(lista, key=lambda o: o.attr)`

✗ Errores típicos

- `return None` **dentro** del `for` con `else` del `if` : solo mira el primer elemento.
- `sorted(lista, key=p.precio)` sin `lambda` → `NameError` .
- `lista.sort()` devuelve `None` : no hagas `lista = lista.sort()` .
- Acceder a `p.__precio` desde fuera de la clase → `AttributeError` : usa el getter.

8 Diccionarios

```python
pelicula = {"titulo": "Inception", "year": 2010}
pelicula["titulo"]                # 'Inception'
pelicula.get("genero", "n/a")     # 'n/a' (sin KeyError)
pelicula["rating"] = 8.8          # añadir / modificar
del pelicula["year"]              # borrar

for clave, valor in pelicula.items():
    print(f"{clave}: {valor}")
```

Contador con diccionario

```python
conteo = {}
for palabra in ["sol", "mar", "sol"]:
    conteo[palabra] = conteo.get(palabra, 0) + 1
# {'sol': 2, 'mar': 1}
```

Lista de diccionarios

```
alumnos = [
    {"nombre": "Ana", "nota": 8},
    {"nombre": "Luis", "nota": 5},
]
for a in alumnos:
    print(a["nombre"], a["nota"])

aprobados = [a for a in alumnos if a["nota"] >= 5]
alumnos.sort(key=lambda a: a["nota"], reverse=True)
```

Agrupar en listas

```
por_curso = {}
for nombre, curso in [("Ana", "1A"), ("Eva", "1A")]:
    por_curso.setdefault(curso, []).append(nombre)
# {'1A': ['Ana', 'Eva']}
```

> ✗ **Errores típicos**
>
> - `dic["clave_que_no_existe"]` → `KeyError`. Usa `.get()` o comprueba con `in`.
> - `for x in dic:` recorre solo las **claves**. Para ambas: `.items()`.
> - Borrar claves mientras recorres el diccionario → `RuntimeError`: recorre `list(dic)`.

9 Comprensiones y lambda

Comprensión de listas

```
#        [EXPRESIÓN  for ELEM in ITERABLE  if FILTRO]
numeros = [1, 2, 3, 4, 5]
cubos = [n ** 3 for n in numeros]              # [1, 8, 27, 64, 125]
pares = [n for n in numeros if n % 2 == 0]     # [2, 4]
largas = [p for p in palabras if len(p) > 3]
etiquetas = ["par" if n % 2 == 0 else "impar" for n in numeros]
```

Comprensión de diccionarios

```
cuadrados = {n: n ** 2 for n in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
caros = {k: v for k, v in precios.items() if v > 100}
```

Equivale a un `for` + `append`, pero en una línea:

```
cubos = []
for n in numeros:
    cubos.append(n ** 3)
```

lambda = función anónima de una línea

```
doble = lambda x: x * 2            # lambda PARAMS: EXPRESIÓN
doble(4)                           # 8

# map: transforma cada elemento
cuadrados = list(map(lambda x: x ** 2, numeros))

# filter: se queda con los que cumplen
pares = list(filter(lambda x: x % 2 == 0, numeros))

# sorted / sort / max / min con key
sorted(palabras, key=lambda p: len(p))
max(alumnos, key=lambda a: a["nota"])
```

Si me piden...	Uso...
Nueva lista transformada "en una línea"	`[f(x) for x in lista]`
Filtrar "en una línea"	`[x for x in lista if cond]`
Usar `map()` con lambda	`list(map(lambda x: ..., lista))`
Usar `filter()`	`list(filter(lambda x: cond, lista))`
Ordenar por un criterio	`sorted(lista, key=lambda x: ...)`

★ FORMA QUE EXIGE EL PROFESOR

- Cubos y filtrado de palabras: **comprensión de listas en una sola línea** (un `for` + `append` funciona, pero no es lo que pide).
- Cuadrados con `list(map(lambda x: x**2, numeros))` (una comprensión funciona, pero no es lo que pide).

✗ Errores típicos

- Olvidar `list()` alrededor de `map` / `filter` : imprime `<map object at ...>` .
- Poner `return` en una lambda → `SyntaxError` .
- El `if` de filtro va al final; el `if/else` de transformar va al principio.

S Métodos de str y f-strings

Las cadenas son **inmutables**: los métodos devuelven una cadena NUEVA. Hay que guardarla: `s = s.strip()` .

Método	Ejemplo	Resultado
`strip()`	`" hola ".strip()`	`'hola'`
`lower()`	`"HoLa".lower()`	`'hola'`
`upper()`	`"hola".upper()`	`'HOLA'`
`capitalize()`	`"hola mundo".capitalize()`	`'Hola mundo'`
`title()`	`"hola mundo".title()`	`'Hola Mundo'`
`split()`	`"a,b,c".split(",")`	`['a', 'b', 'c']`
`join()`	`"-".join(["a", "b"])`	`'a-b'`
`replace()`	`"casa".replace("a", "o")`	`'coso'`

`isdigit()`	`"123".isdigit()` / `"-3".isdigit()`	`True` / `False`
`startswith()`	`"python".startswith("py")`	`True`
`endswith()`	`"foto.png".endswith(".png")`	`True`
`find()`	`"hola".find("l")` / `find("z")`	`2` / `-1`
`count()`	`"banana".count("a")`	`3`
`len()`	`len("hola")`	`4`
`in`	`"ol" in "hola"`	`True`
slicing	`"python"[0:2]` / `[::-1]`	`'py'` / `'nohtyp'`

f-strings

```
nombre, precio, n = "Ana", 3.14159, 7
f"Hola, {nombre}"            # 'Hola, Ana'
f"{precio:.2f} €"            # '3.14 €'  (2 decimales)
f"{n:03d}"                   # '007'     (rellena con ceros)
f"{nombre:<10}|"             # 'Ana       |' (alinear izq.)
f"{nombre:>10}|"             # '       Ana|' (alinear der.)
f"{0.256:.1%}"               # '25.6%'
f"{1234567:,}"               # '1,234,567'
f"{precio=}"                 # 'precio=3.14159' (depurar)
f"Total: {a + b}"            # se pueden poner expresiones
```

Recetas

```
texto = input("Nombre: ").strip().title()   # limpiar entrada
vocales = sum(1 for c in texto.lower() if c in "aeiou")
palabras = frase.split()          # separa por espacios
es_palindromo = t == t[::-1]
s[0]   s[-1]   s[1:4]   s[:3]   s[3:]
```

L Métodos de list

Método	Ejemplo (`l = [3, 1, 2]`)	Efecto / resultado
`append(x)`	`l.append(5)`	`[3, 1, 2, 5]` (al final)
`insert(i, x)`	`l.insert(0, 9)`	`[9, 3, 1, 2]`
`remove(x)`	`l.remove(1)`	borra el **primer** 1 (ValueError si no está)
`pop()`	`l.pop()` / `l.pop(0)`	quita y **devuelve** el último / el de índice 0
`sort()`	`l.sort()` / `l.sort(reverse=True)`	ordena la propia lista, devuelve `None`
`sorted()`	`nueva = sorted(l)`	lista NUEVA ordenada; `l` no cambia
`index(x)`	`l.index(2)`	`2` (posición; ValueError si no está)
`count(x)`	`l.count(3)`	`1`
`in`	`2 in l`	`True`
`min / max`	`min(l)` , `max(l)`	`1` , `3`
`sum / len`	`sum(l)` , `len(l)`	`6` , `3`

`reverse()`	`l.reverse()`	invierte la propia lista
`extend()`	`l.extend([7, 8])`	añade varios

Slicing `lista[inicio:fin:paso]`

```
l = [10, 20, 30, 40, 50]
l[0]       # 10        primero
l[-1]      # 50        último
l[1:3]     # [20, 30]  (el fin NO entra)
l[:2]      # [10, 20]  los 2 primeros
l[2:]      # [30, 40, 50]
l[::2]     # [10, 30, 50]
l[::-1]    # [50, 40, 30, 20, 10] invertida
copia = l[:]          # copia (no l2 = l)
```

Otras útiles

```
media = sum(l) / len(l)
if l:                      # ¿no está vacía?
    ...
for i, x in enumerate(l, start=1): ...
for a, b in zip(nombres, notas): ...
numeros = [int(x) for x in "1 2 3".split()]
```

D Métodos de dict

Ejemplos con `d = {"a": 1, "b": 2}`

Método	Ejemplo	Resultado / efecto
`get(k, def)`	`d.get("a")` / `d.get("z", 0)`	`1` / `0` (sin `KeyError`)
`keys()`	`list(d.keys())`	`['a', 'b']`
`values()`	`sum(d.values())`	`3`
`items()`	`for k, v in d.items():`	pares `('a', 1)`, `('b', 2)`
`pop(k)`	`d.pop("a")`	quita la clave y devuelve `1`
`update()`	`d.update({"c": 3})` / `d.update(c=3)`	añade o sobrescribe varias
`in`	`"a" in d`	`True` (mira las **claves**)
`del`	`del d["b"]`	borra la clave (KeyError si no está)
añadir / cambiar	`d["z"] = 26`	crea o reemplaza
`len()`	`len(d)`	número de claves
`setdefault()`	`d.setdefault("l", []).append(1)`	crea con valor por defecto si no existe
clave del valor máximo	`max(d, key=d.get)`	`'b'`
ordenar por valor	`sorted(d.items(), key=lambda kv: kv[1])`	lista de tuplas ordenada

T Conversión de tipos

Conversión	Ejemplo	Resultado
`int()`	`int("42")` / `int(3.9)`	`42` / `3` (trunca)
`float()`	`float("3.5")`	`3.5`
`str()`	`str(42)`	`'42'`
`bool()`	`bool("")` / `bool("no")`	`False` / `True`
`list()`	`list("abc")`	`['a', 'b', 'c']`
`tuple()` / `set()`	`set([1, 1, 2])`	`{1, 2}` (sin repetidos)
`round()`	`round(2.678, 2)`	`2.68`
`type()`	`type(3.0)`	`<class 'float'>`
`isinstance()`	`isinstance(5, int)`	`True`

```python
edad = int(input("Edad: "))          # input SIEMPRE da str
precio = float(input("Precio: "))

# int("3.5") -> ValueError; usa float() primero
# int("hola") -> ValueError

try:                                 # validar con try/except
    n = int(input("Número: "))
except ValueError:
    print("Eso no es un número")

print("Tienes " + str(edad) + " años")   # concatenar: str()
print(f"Tienes {edad} años")             # mejor f-string
```

J Java → Python

Java	Python
`&&` / `||` / `!`	`and` / `or` / `not`
`this`	`self` (primer parámetro, explícito)
constructor `Clase(...)`	`def __init__(self, ...):`
`switch` / `default`	`match` / `case _:`
`i++` / `i--`	`i += 1` / `i -= 1`
`null`	`None`
`true` / `false`	`True` / `False` (mayúscula)
`System.out.println(x)`	`print(x)`
`Scanner sc = new Scanner(System.in)`	`input("mensaje")` (devuelve str)
`private int x;`	`self.__x` (dos guiones bajos)
`toString()`	`__str__(self)`

`for (int i = 0; i < n; i++)`	`for i in range(n):`
`for (String s : lista)`	`for s in lista:`

Java	Python
`{ }` para bloques	`:` + sangría de 4 espacios
`;` al final	nada
`else if`	`elif`
`int x = 5;`	`x = 5` (sin tipo)
`new Perro("Rex")`	`Perro("Rex")` (sin new)
`ArrayList<T>`	`[]` (list)
`HashMap<K,V>`	`{}` (dict)
`lista.size()` / `s.length()`	`len(lista)` / `len(s)`
`lista.add(x)` / `map.put(k, v)`	`lista.append(x)` / `d[k] = v`
`s.equals(t)`	`s == t`
`class B extends A`	`class B(A):`
`super(x)`	`super().__init__(x)`
`try { } catch (E e) { }`	`try: ... except E as e: ...`
`public static void main`	`if __name__ == "__main__":`
`// comentario` `/** doc */`	`# comentario` `"""docstring"""`

★ Resumen: formas que exige el profesor

Si el enunciado dice...	Tienes que usar...	Aunque funcione, NO vale...
"match" / menú de días	`match` + `case` + `case _`	cadena de `if/elif`
clasificar rangos dentro de match	guardas `case p if ...`	if dentro del case
rango "entre A y B"	`A <= x <= B`	`x >= A and x <= B`
"cláusula de guarda"	`if error: ... return` al principio	if/else anidado
"sin usar sum()"	bucle `for` + acumulador	`sum()`
"usando range()"	`range(inicio, fin, paso)`	while con contador
numerar desde 1	`enumerate(lista, start=1)`	contador manual `i += 1`
"usa el else del bucle"	`for ... break` / `else:`	variable bandera
"while True con break"	`while True:` + `break`	while con condición
"usa continue"	`continue`	`if n % 3 != 0: print(n)`
"en una sola línea" / "list comprehension"	`[expr for x in lista if cond]`	for + append
"retorna"	`return`, sin `print` dentro	print del resultado
"retornos múltiples (tupla)"	`return a, b` + `x, y = f()`	devolver una lista

"utilizando *args / **kwargs"	`*args` / `**kwargs`	pasar una lista / un dict
"Type Hints" y "Docstring"	`(c: float) -> float` + `"""..."""`	solo comentario #
"decorador" / "sintaxis @"	`@anunciar_ejecucion`	`f = anunciar_ejecucion(f)`
"map() junto con lambda"	`list(map(lambda x: x**2, numeros))`	comprensión o for
"función recursiva"	se llama a sí misma + caso base	bucle
atributos "privados"	`self.__atributo`	`self._atributo` o público
"getter" que informa del estado	método con `return` de un str	método que imprime
pruebas / "bloque principal"	`if __name__ == "__main__":`	código suelto al final
parámetro lista/dict por defecto	`=None` e inicializar dentro	`=[]` o `={}`